

Extremely Detailed Developer & User Flow Architecture

This document serves as the absolute, granular source of truth for the `SUBDOMAIN_SAAS_DEMO` request lifecycles. It includes every validation gate, fallback mechanism, TTL config, rate limit parameter, and raw SQL/Redis operation triggered during normal operation.

1. Subdomain Request Delivery (Visitor Flow)

When an external visitor or browser requests `http://<tenant>.lvh.me:3000` (or the production equivalent).

1.1 proxy.ts (Edge Interception)

- **Trigger:** Next.js middleware fires before the Next.js router initializes.
- **Exclusion Rules:** Implicitly ignores paths starting with `/_next`, `/api`, and `/favicon.ico`.
- **Hostname Extraction:**
 1. Extracts the `Host` header.
 2. Strips local dev ports (`:3000` or `:8081`).
 3. Splits the string by `.` to isolate the subdomain string.
 4. Ignores `www` or root level top-level-domains.
- **Rewriting:** If a subdomain (e.g., `cafe`) is resolved, silently performs an internal Next.js rewrite from `/` to `/sites/cafe` . The URL in the user's browser does not change.

1.2 Tenant Rendering (`app/sites/[tenant]/page.tsx`)

- **Trigger:** Request hits the page component.
- **Action:** A synchronous SQLite query executes to lookup the tenant by Name (`SELECT * FROM tenants WHERE name = ?`).
- **State Switch:**
 - **Condition A (Idle):** If `generation_status` is `idle` or `failed` , Next.js renders the fallback standard React template containing a Hero section, Features section, and Footer.
 - **Condition B (Generating/Pending):** Renders a skeleton UI with a "Building Your Site" loading spinner.
 - **Condition C (Success):** If the site is built, renders a sandboxed `<iframe>` pointing to `/api/sites/<tenant>` . The `sandbox` attribute ONLY uses `allow-scripts` (intentionally omitting `allow-same-origin` to isolate the payload completely).

1.3 Secure Payload Delivery (`/api/sites/[tenant]/route.ts`)

- **Trigger:** The iframe requests the raw HTML.
 - **Cache Lookup:** Checks the in-memory cache (`cache.get(tenant)`). If the cache is valid (TTL is 5 minutes maximum), skips the database.
 - **Cache Miss:** Queries `SELECT html_content FROM tenants` . Populates `cache.set()` .
 - **Security Headers Injection:** The response injects the strictest possible Content Security Policies (CSPs) before returning the raw HTML:
 - `connect-src 'none'` : Prevents data exfiltration hooks via `fetch()` or `XHR` .
 - `object-src 'none'` : Blocks Flash/Java/Plugin embedding.
 - `base-uri 'none'` : Prevents injected base paths.
 - `frame-ancestors 'self'` : Disallows the raw HTML from being framed by external sites (clickjacking protection).
-

2. Tenant Registration Flow

Occurs within the management dashboard logic.

2.1 Initialization

- **Action:** Dashboard UI posts raw `name` and `title` to `/api/tenants`.
- **Authentication:** `getSessionUserId()` extracts the current session token (mocked locally for demo users).
- **Initialization:** Executes `INSERT OR IGNORE INTO users` to ensure a foreign key relations target exists for the user.

2.2 Persistence

- **Action:** `INSERT INTO tenants (id, user_id, name, title)`.
 - **Database Level Enforcement:** There is a strong `UNIQUE` database constraint on the `name` column. If a duplicate string exists, sqlite rejects the write.
 - **Response:** The tenant is instantiated with a default `generation_status` set to `idle`. Version integer sits at `0`. HTTP 201 is returned.
-

3. The 7-Gate Site Generation API Flow

This is the most critical ingestion point when a user requests an AI-generated site. Location:

`/api/stitch/generate/route.ts`

1. **Gate 1 - Payload Validation:** Asserts that `tenantId` and `prompt` strings exist in the JSON payload body.
2. **Gate 2 - Ownership Authorization:** Queries the `tenants` table matching `tenantId`. Asserts that `tenant.user_id === session.user_id`. Failure throws a 403 or 404 security rejection.
3. **Gate 3 - Prompt Size Constraint:** Asserts that the string length of the user's prompt strictly `<= 2000` characters. Exceeding throws a 400.
4. **Gate 4 - Rate Limiting:**
 - Evaluates per-user and per-IP thresholds by counting records in the `usage_tracking` database within a 1-hour rolling window.
 - Limit: 10 generations per Hour per User ID.
 - Limit: 20 generations per Hour per raw IP Address.
 - Rejection throws HTTP 429 Too Many Requests.
5. **Gate 5 - Queue Backpressure Lock:**
 - Assesses the length of the Upstash Redis queue `stitch:job_queue`.
 - If `queueLength > 100`, Rejects the payload with HTTP 503 (Service Unavailable) to prevent worker OOM flooding.
6. **Gate 6 - Job Registration:**
 - Database creates a record in `stitch_jobs` mapped to the `tenantId`. `status` is set to `pending`.
7. **Gate 7 - Queue Ingestion (Retry Safety):**
 - Invokes `safeEnqueue()` -> `LPUSH` payload onto Redis.
 - **Retry Fallback:** If the network request to Redis fails, it will exponentially backoff and retry up to 3 times.

- **Critical Failure:** If all 3 Redis `LPUSH` attempts fail, the DB record `status` is overridden to `failed` and connection aborted.
 - **Success Resolution:** Updates `tenants` table `generation_status = 'pending'` . Returns HTTP 200 containing the Job ID, allowing the browser to begin interval polling.
-

4. The Background Cron Worker Pipeline

An external automated polling trigger hitting `/api/cron/process-jobs/route.ts` every ~5 seconds. It processes through 3 strict phases before returning.

4.1 Phase 1: Recovery Loop (`recoverMissingJobs()`)

- Scans the DB for `status = pending` jobs where `locked_at` is NULL and time created > 30 seconds ago.
- Asserts that `retry_count < max_retries` (prevents infinite loop processing of poison messages).
- Any job matching this gets force re-enqueued to the Redis queue effectively solving any Queue/DB disconnects.

4.2 Phase 2: Stale Lock Cleanup (`cleanupStaleJobs()`)

- Looks for crashed jobs: `status = processing` but `locked_at` is older than 5 minutes.
- **Queue Cleanup:** FIRST explicitly calls `acknowledgeJob()` to perform an `LREM` removing the crashed job from the specific "processing queue".
- **Restart:** Updates database `locked_at` to NULL and pushes the job back to the main queue safely.

4.3 Phase 3: SDK Integration & Resolution (`processNextJob()`)

Worker processes up to `CRON_BATCH_SIZE` (default 5) in sequence.

1. **Queue Retrieval:** Executes Redis `LMOVE` . This synchronously moves the Job from the `main_queue` directly onto a `processing_queue` atomically. No data is lost if the pod shuts down abruptly.
2. **Atomic Lock Contention:** Attempts to lock the worker in SQLite: `UPDATE stitch_jobs SET locked_at = NOW() WHERE id = ? AND locked_at IS NULL`
 - If `changes() === 0` , it means another cron execution thread stole the lock. The job is immediately aborted and NOT acknowledged in Redis (another worker owns it).
3. **Execution:**
 - Fires `stitchService.generate()` , awaiting response from the Google SDK.
4. **Validation Pipeline (`html-validator.ts`):**
 - Size Check: Evaluates raw string byte length is `< 2MB` . If it fails this, throws limit error.
 - Sanitization Hook: Passes raw HTML through `sanitize-html` . Completely strips any `<script>` tags, and malicious tag attributes (`onLoad` , `onClick`).
5. **Fallback Cascade (If SDK format errors out but doesn't timeout):**
 - Tier 1: Pull the last known good snapshot from `stitch_history` .
 - Tier 2: Pull the current `html_content` live on the tenant record.
 - Tier 3: Hardcoded emergency `<div class="p-8">Generation failed...</div>` .
6. **Atomic Transaction Commit (BEGIN...COMMIT):**
 - If everything succeeds, it initiates an exact DB transaction:

- `UPDATE tenants` sets `html_content` , bumps `version = version + 1` , logs `new stitch_project_id` .
 - `INSERT INTO stitch_history` -> Captures full historical payload blob.
 - `UPDATE stitch_jobs` sets `status = success` , logs time.
 - `INSERT INTO usage_tracking` (For billing/rate limit monitoring).
 - `INSERT INTO job_metrics` logs latency and `queue_wait_ms` .
- If successful, calls `acknowledgeJob()` to `LREM` delete the item from Redis permanently.
 - Clears `cache.invalidate()` allowing the visitor UI to instantly update.
-

5. Branch Operations: Refinement and Rollback

5.1 Refinement SubFlow

- Validates the user identically to Generation (All 7 gates apply).
- **Primary Divergence:** Enforces Gate 2.5: Queries database `SELECT stitch_project_id FROM tenants` . It will explicitly reject the request with HTTP 400 if the tenant has not had an initial site generated.
- Maps to the SDK's `refine()` function rather than `generate()` , pushing delta-patches to the active screen context rather than building from scratch.

5.2 Rollback SubFlow

- The user requests reversion to a specific numeric history state (e.g., Target Version: `2`).
 - Target Lookup: Scans `stitch_history` records matching the exact version numeral and tenant id.
 - DB Transaction:
 1. Copies the historical `html_content` payload over the live `html_content` value in `tenants` .
 2. Bumps the master version variable (`version = 5 + 1 = 6`).
 3. Appends a *new* historical snapshot to `stitch_history` confirming version 6 is identical to state version 2. (This guarantees linear timeline logging, never destroying history).
 - Instantly flushes localized Caches.
-

End of document. Architecture validates against v5 Codebase State with complete implementation loops.